

Exercise Sheet 6

Lab Sessions: June 11/12, 2026

1 Preparation

For this exercise, we provide files in the [RepEng FIMGit Repository](#) (directory `LabSession6/`). Update your local repository that you cloned in the first lab session:

1. Open a terminal in your local repository.
2. Pull the latest changes using `git pull`.

2 Challenges in Reproducing Binary Builds

So far, we have worked primarily with interpreted programming languages such as Python. In this exercise, we will focus on compiled binaries instead. Re-creating binaries from source code might seem an easy task, but there are many factors that affect reproducibility. We discuss some of these factors using a simple C program as an example.

2.1 Setting up Docker

First, we need to set up a Docker container:

- (a) Enter the directory `LabSession6/task2/` in the [RepEng repository](#).
- (b) Build a Docker image using the provided `Dockerfile`.
- (c) Run an interactive container from that image.

2.2 Comparing different Compilers

Inside the container, we compare two binaries created with different compilers from the same source code by following these steps:

- (a) Enter the directory `task2` and inspect the source code in `hello.c`.
- (b) Compile `hello.c` using the `gcc` compiler.

```
| user@container$ gcc hello.c -o hello-gcc
```

- (c) Compile `hello.c` using the Clang compiler.

```
| user@container$ clang hello.c -o hello-clang
```

- (d) Compare the functionality of the programs by executing them. Are they functionally equivalent?
- (e) Compare the program binaries. Are they bitwise identical?
- (f) Discuss, what your results mean for reproducibility.

2.3 Comparing different Compile Options

Compiler optimizations improve performance and/or code size at the expense of compilation time and possibly the ability to debug the program. We now compare different compiler optimizations:

- (a) Compile `hello.c` with two different levels of optimization using `gcc`.

Hint: You can find a list of optimization options in [gcc's online documentation](#).

- (b) Compare the functionality of the programs by executing them. Are they functionally equivalent?
- (c) Compare the program binaries. Are they bitwise identical?

2.4 Debug Information and Preprocessors

Next, we look at the impact of different debug compile flags and preprocessor macros:

- (a) Compile `hello.c` with debug information using `gcc`.

```
| user@container$ gcc -g hello.c -o hello-debug
```

- (b) Compare the binaries `hello-gcc` and `hello-debug`. Are they bitwise identical?
- (c) Change the preprocessor macro in `hello.c` to output the time at which the preprocessor is being run.

The following command line session shows an example of the expected output:

```
| Terminal Session bash
| user@container$ gcc hello.c -o hello && date +%T && sleep 5 \
|                               && ./hello
| 11:28:40
| 11:28:40: Hello World
```

It compiles `hello.c` using `gcc`, shows the current system time (`date +%T`), waits for 5 seconds (`sleep 5`), and finally executes the compiled binary. The system time (first line of the output) should be (almost) identical to the timestamp printed by the `hello` binary (second line of the output).

Note: Using this chain of commands, you might see a difference in the timestamp produced by `date +%T` and the `hello` binary, depending on how long building takes. For a simple program such as this one, the timestamps will likely be identical or at least very close.

Hint: You can find a list of preprocessor macros in the [gcc documentation](#)

- (d) Compile `hello.c` using the `gcc` compiler. Then compile `hello.c` using the `gcc` compiler again, but use a different output path.
- (e) Compare the functionality of the programs by executing them. Are they functionally equivalent?
- (f) Compare the program binaries. Are they bitwise identical?
- (g) Change the preprocessor macro in `hello.c` that it prints the source file path, e.g.:

Terminal Session

bash

```
user@container$ gcc hello.c -o hello-file
user@container$ ./hello-file
hello.c: Hello World
```

- (h) Compile `hello.c` from within the `task2/` directory, as well as from the parent directory (`pwd` prints the current working directory):

Terminal Session

bash

```
user@container$ pwd
/home/repro/task2
user@container$ gcc hello.c -o hello-file
user@container$ cd ..
user@container$ pwd
/home/repro/
user@container$ gcc task2/hello.c -o hello-file2
```

- (i) Check whether the two binary programs you just compiled are functionally equivalent and/or bitwise identical.
- (j) Discuss, what your observations mean for reproducibility.

3 Reproducible Toolchains: Compiling Python

For some reproducible builds, building software from the source code is necessary, for instance, because the binaries are not readily available or because some modifications to the software are made. As an exercise, we build Python from source.

- (a) Inside the Docker container from Exercise 2, create a new directory named `task3` and enter it.
- (b) Download [Python 3.12.3](#) from the official release server:

```
user@container$ curl -O \
    "https://www.python.org/ftp/python/3.12.3/Python-3.12.3.tgz"
```

- (c) Unpack the source code.

```
| user@container$ tar -xvf Python-3.12.3.tgz
```

- (d) Configure the build “out of source”.

```
| user@container$ mkdir build
| user@container$ cd build
| user@container$ ../Python-3.12.3/configure \
|     --prefix=$HOME/Python-3.12.3-custom
```

- (e) Start the build process:

```
| user@container$ make -j $(nproc)
```

Hints:

- `-j $(nproc)` configures `make` to use all available cores on your system (if possible). This usually speeds up the build process but increases system load. To configure the number of parallel threads used in the build process, replace `$(nproc)` with the desired number.
- Building Python will take a while. You can continue working on the next exercises while you wait.

- (f) Discuss why an “out of source” or “out of tree” build is considered good practice.

4 ReproTest

A central aspect in ensuring reproducible builds is testing them in different environments. [ReproTest](#) is a tool that supports this process by building a program twice in different, simulated environments, and checking whether identical binaries have been produced.

For this exercise, we will use a different Docker container.

4.1 Extending the Dockerfile

First, extend the Dockerfile located at `LabSession6/task4/Dockerfile` in the RepEng repository to contain ReproTest:

- (a) Enter the directory `LabSession6/task4`.
- (b) Extend the `Dockerfile` so that ReproTest is automatically installed and necessary artifacts are added to the container. Additional necessary dependencies are `disorderfs` and `faketime`.
- (c) Build a Docker image using the `Dockerfile`.

```
| user@host$ docker build -t lab6-4 .
```

- (d) Run a container with that image in privileged interactive mode.

```
| user@host$ docker run -it --privileged lab6-4
```

Note: Generally, avoid running Docker containers in privileged mode due to security reasons. However, it is necessary for reprotest to function properly.

4.2 Using ReproTest

Next, we check the reproducibility of our code with ReproTest:

- (a) Test `hello-line.c` using ReproTest:

```
| user@container$ reprotest 'gcc hello-line.c -o hello-line' hello-line
```

What does the output of ReproTest tell you about the reproducibility of this gcc build?

- (b) When compiling the code in debug mode, ReproTest will discover differences in the binaries:

```
| user@container$ reprotest 'gcc -g hello-line.c -o hello-line' \
hello-line
```

Discuss the cause for these differences and where they appear in the binaries, using the terminology for memory management in C, as shown in Figure 1.

- (c) Change the preprocessor macro in the code, so that reproduction fails according to ReproTest, even without using debug mode.



Memory memorizer

Stack

This is the section of memory used for **local variable storage**. Every time you call a function, all of the function's local variables get created on the stack. It's called the *stack* because it's like a stack of plates: variables get added to the stack when you enter a function, and get taken off the stack when you leave. Weird thing is, the stack actually works upside down. It starts at the top of memory and **grows downward**.

Heap

This is a section of memory we haven't really used yet. The heap is for **dynamic memory**: pieces of data that get created when the program is running and then hang around a long time. You'll see later in the book how you'll use the heap.

Globals

A global variable is a variable that lives outside all of the functions and is visible to all of them. Globals get created when the program first runs, and you can update them freely. But that's unlike...

Constants

Constants are *also* created when the program first runs, but they are stored in **read-only** memory. Constants are things like *string literals* that you will need when the program is running, but you'll never want them to change.

Code

Finally, the code segment. A lot of operating systems place the code right down in the lowest memory addresses. The code segment is also read-only. This is the part of the memory where the actual assembled code gets loaded.

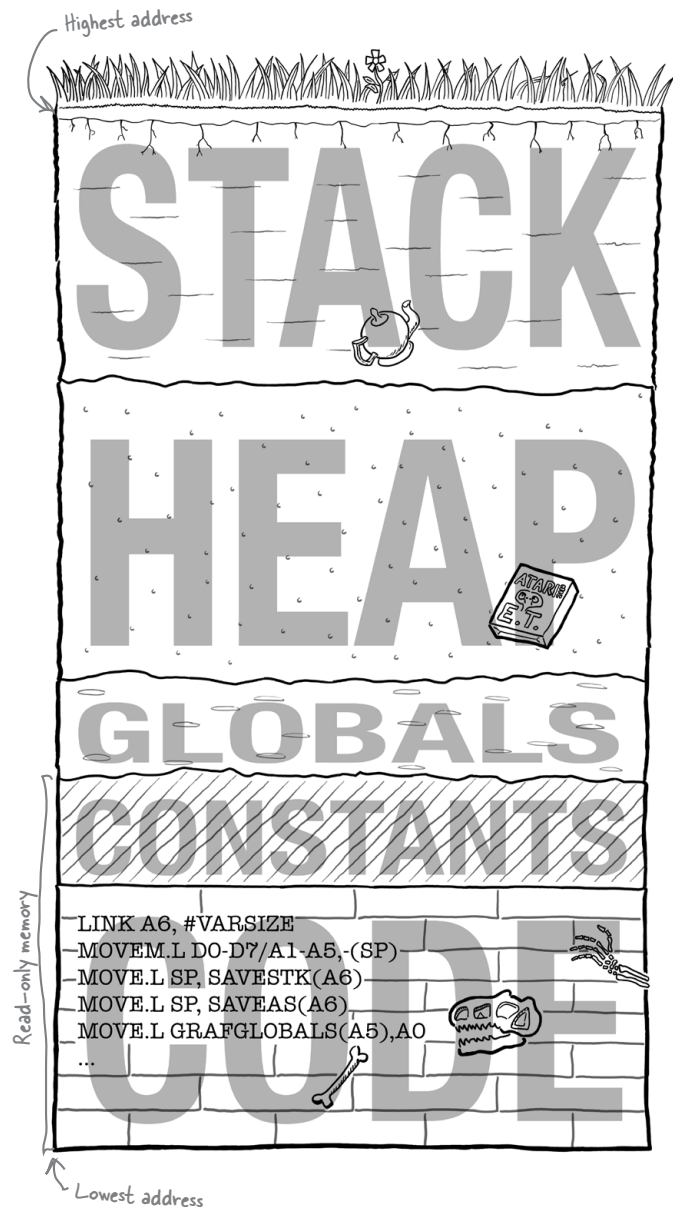


Figure 1: Memory management in C. Source: “Head first C”.

5 Make

Make is a tool to automate the generation of files from source. In this exercise, we create an instruction file for Make, called a `makefile`.

- (a) In the RepEng repository, change the directory to `LabSession6/task5/`.
- (b) Build the Docker image from the Dockerfile and run it interactively.
- (c) Inspect the Python code and the LaTeX file. Think about their dependencies and create a `makefile` with all these dependencies.

Hint: The PDF from a tex file with “quiet” output can be generated with the following command:

```
$ latexmk -pdf -interaction=batchmode -halt-on-error <tex-file>
```

`latexmk` is a more convenient solution to compile LaTeX, as `pdflatex` might need to be executed (manually) multiple times to resolve references, whereas `latexmk` automatically executes `pdflatex` as often as necessary.

- (d) Create a PDF file from `sin.tex` by running the `makefile`.
- (e) Copy the created PDF file from the container to the host file system by running the following command outside the container (assuming the filename is `sin.pdf`):

```
user@host$ docker cp <container_ID>:/home/repro/sin.pdf .
```

Afterwards, inspect the PDF file to ensure that it has been generated correctly.

- (f) Execute the script `csv_noisy_sin.py` again. Which commands are executed again when applying the `makefile`?
- (g) Change the sample size in `gen_df_sin.py` to 2048. Which commands are executed again when applying the `makefile`?
- (h) Change the noise strength in `csv_noisy_sin.py` to 0.4. Which commands are executed again when applying the `makefile`?

6 Reproducible Builds (Antwort-Wahl-Verfahren)

In this exercise, each question has exactly one correct answer. Mark the correct answer.

- (a) Consider the following C code snippets. We want to achieve bitwise identical builds when compiling the **same source code** on **different days** and potentially in **different build directories**.

Program 1:

```
#include <stdio.h>
int main(void) {
    printf("%s\n", "Hello World");
    return 0;
}
```

Program 2:

```
#include <stdio.h>
int main(void) {
    printf("File: %s\n", __FILE__);
    return 0;
}
```

Program 3:

```
#include <stdio.h>
int main(void) {
    printf("Built: %s\n", __TIME__);
    return 0;
}
```

Program 4:

```
#include <stdio.h>
int main(void) {
    printf("Line: %d\n", __LINE__);
    return 0;
}
```

How many of the snippets above allow for a bitwise identical build?

- ☐ 0 ☐ 1 ☐ 2 ☐ 3 ☐ 4

- (b) Consider the following Makefile for our pplease experiment pipeline. It runs the bash dispatcher, plots the results, and compiles the final LaTeX paper:

Makefile

```
# Declare 'all' and 'clean' as phony targets. This ensures these
# commands run even if a file named 'all' or 'clean' exists.
.PHONY: all clean

# Default target: builds the final document.
all: experiment.pdf

# Compile experiment.pdf using latexmk (c.f. Exercise 5 (c))
experiment.pdf: experiment.tex results/chart.pdf
    latexmk -pdf -interaction=nonstopmode -halt-on-error experiment.tex

# Generate the chart from the CSV data.
results/chart.pdf: generate_chart.py results/results.csv
    python3 generate_chart.py results/results.csv 10 results/chart.pdf

# Run the experiment script to generate raw data.
results/results.csv: run_experiment.sh pplease.py pplease_split.py \
    pplease_stats.py recipe.txt
    bash run_experiment.sh recipe.txt 10 42 make_run

# Remove all generated PDFs, data files, and LaTeX log artifacts.
clean:
    rm -f experiment.pdf results/chart.pdf
    rm -f results/results.csv results/polite_*.txt results/stats_*.txt
    latexmk -c
```

Assume that you just ran `make all` successfully, meaning the project is completely up-to-date. These files are now in the directory:

Terminal Session

bash

```
user@container$ ls -l
-rwxr-xr-x 1 repro repro    306 May 29 13:04 doAll.sh
-rw-r--r-- 1 repro repro   1452 May 29 13:17 experiment.aux
-rw-r--r-- 1 repro repro  10548 May 29 13:17 experiment.fdb_latexmk
-rw-r--r-- 1 repro repro  10680 May 29 13:17 experiment.fls
-rw-r--r-- 1 repro repro  14667 May 29 13:17 experiment.log
-rw-r--r-- 1 repro repro   1041 May 29 13:17 experiment.out
-rw-r--r-- 1 repro repro 102461 May 29 13:17 experiment.pdf
-rw-r--r-- 1 repro repro   2055 May 29 12:51 experiment.tex
-rw-r--r-- 1 repro repro   2639 May 29 13:17 generate_chart.py
-rw-r--r-- 1 repro repro    560 May 29 13:16 makefile
-rw-r--r-- 1 repro repro    772 May 29 12:51 pplease.py
-rw-r--r-- 1 repro repro    388 May 29 12:51 pplease_split.py
-rw-r--r-- 1 repro repro   1172 May 29 12:51 pplease_stats.py
-rw-r--r-- 1 repro repro    519 May 29 12:51 recipe.txt
drwxr-xr-x 1 repro repro    504 May 29 13:17 results
-rwxr-xr-x 1 repro repro   2499 May 29 13:04 run_experiment.sh
```

Afterwards, you notice a typo in the legend of your chart and modify the script `generate_chart.py`. If you run `make all` again after saving `generate_chart.py`, which targets (files) will Make recreate?

- ☐ Only `results/chart.pdf`
- ☐ `results/chart.pdf` and `experiment.pdf`
- ☐ All targets (except `clean`)
- ☐ None of the targets